

Project

Ekaterina Salova ID – 121132

1 Program

Môj kód najprv triedi zostupne, potom vo funkcii *rozvrh* začneme od najbližšieho termínu, kontrolujeme, či daná práca bude vykonaná načas a aký zisk prinesie. Túto funkciu som vytvorila na základe uvedenej funkcie. Na konci si zapíše úlohy potrebné na splnenie a celkový zisk.

Môj program má 1 for, 4 forfor a 1 forforfor, pozrime sa na každý z nich, aby sme pochopili zložitosť programu.

1.

```
for (i=0; i<n-1; i++) {           // n-1 krat
    for (j=0; j<n-i-1; j++) {     // n-i-1 krat
        if (profit[j] < profit[j+1]) { // n-i-1 krat
            swap(j, j+1);
        }
    }
}
```

Zložitosť = $O(n-1) \times O(n-i-1) = O(n^2 - in - n - n + i + 1) = O(n^2 - (i+2)n + i + 1) \approx O(n^2)$

2.

```
for (i=2; i<=n; i++) {           // n-1 krat; for
    for (j=0; j<*sizeJ; j++) {     // sizeJ krat
        K[j] = J[j];
    }
    K[*sizeJ] = i;
    sizeK = j+1;
    // K=J with i added according to nondecreasing value
    for(a=0; a<sizeK-1; a++) {     // sizeK-1 krat
        for(b=a+1; b<sizeK; b++) { // sizeK-a-1 krat
            if (deadline[K[a]-1] > deadline[K[b]-1]) {
                j = K[a];
                K[a] = K[b];
                K[b] = j;
            }
        }
    }
}

if (feasible(K, sizeK, deadline)) { // if (K is feasible)
```

```
    for (i=0; i<size; i++) {       // sizeK krat
```

```
        for(j=0; j<sizeK; j++) {   // sizeK krat
            J[j] = K[j];           // J = K;
        }
    }
```

$$\begin{aligned} \text{Zložitosť} &= O(n-1) \times (O(\text{size}f) + (O(\text{size}K-1) \times O(\text{size}K-a-1)) + O(\text{size}K) + \\ &O(\text{size}K)) \approx O(n-1) \times (O(n) + O(\text{size}K^2 - (a+2)\text{size}K + a+1) + O(n)) \approx O(n) \times \\ &(O(n) + O(n^2) + O(n)) \approx O(n) \times O(n^2) \approx O(n^3) \end{aligned}$$

3.

```
for (i=0; i<sizeJ; i++) { // sizeJ krat
```

Zložitosť = $O(\text{size}f) \approx O(n)$

Celkova zložitosť programy = $O(\text{size}f) \approx O(n^3) + O(n^2) + O(n) \approx O(n^3)$

2 Program

Môj program rieši problém optimálneho rozvrhnutia práce pomocou diskrétnej množiny (Disjoint Set). Úlohy sú zoradené v zostupnom poradí podľa zisku. Následne sú pomocou štruktúry Disjoint Set priradené k posledným možným slotom, ktoré neprekračujú svoj termín. Výsledkom je optimálne poradie úloh na maximalizáciu zisku, ktoré program zobrazí.

Môj program má 5 for a 1 qsort, pozrime sa na každý z nich, aby sme pochopili zložitosť programu.

1.

```
55 for (i=0; i<n; i++) { // n krat
```

Zložitosť = $O(n)$; $\theta(n)$

2.

```
62 for (i=0; i<n; i++) { // n krat
```

Zložitosť = $O(n)$; $\theta(n)$

3.

```
69 qsort(tasks, n, sizeof(Task), compare);
```

Zložitosť = $O(n \log n)$; $\theta(n \log n)$

4.

```
30 for (i=0; i<=d; i++) { // d krat
```

Zložitosť = $O(d) \approx O(n)$; $\theta(d)$

5.

```
35 for (i=0; i<n; i++) { // n krat
36 slot = small(tasks[i].deadline, sets); // log m krat
```

```
18 int small(int S, int sets[]) { // log m krat
19 if (sets[S] == S) {
20 return S;
21 }
22 return sets[S] = small(sets[S], sets);
```

Zložitosť = $O(n) \times (O(\log n) + O(\log n)) = O(n \log n)$;

Ak $d > n$, teda ak termíny úloh presahujú počet úloh, v praxi bude maximálny dostupný slot obmedzený na n (počet úloh).

Ak $d < n$, využívame termíny úloh ako obmedzenie, a počet dostupných slotov sa tým pádom rovná d (najvyššiemu termínu).

Z tohto vyplýva, že minimálna z týchto dvoch hodnôt, $m = \min(d, n)$, určuje zložitosť operácií **find** a **union**.

$\theta(n \log n)$, ak $n > d$, inak $\theta(n \log d)$ znamená $\theta(n \log m); m = \min(n, d)$

6.

```
75 for (i=0; i<sizeJ; i++) { // sizeJ krat
```

Zložitosť = $O(\text{sizeJ}) \approx O(n)$; $\theta(d)$

Celková zložitosť programy = $O(n) + O(n) + O(n) + O(n \log n) + O(n \log n) \approx O(n \log n)$

3 Program

Program rieši problém minimálnych nákladov na zlúčenie n súborov do jedného pomocou algoritmu na báze min-haldy (min-heap). Počíta celkové náklady zlúčenia ako súčet veľkostí vždy dvoch najmenších súborov, pričom výsledné zlúčené súbory sa opäť pridávajú do haldy. Cieľom je minimalizovať celkový počet presunov a zlúčení. Na záver program vypíše celkové náklady zlúčenia pre rôzne vstupné príklady.

Môj program má for, 2 while a min, pozrime sa na každý z nich, aby sme pochopili zložitosť programu.

1.

```
48 for (i=(n/2)-1; i>=0; i--) {
49     min_heap(files, n, i);
50 }
```

Zložitosť = $O(\log n)$

2.

```
54 a = minimum(files, &n);
55 b = minimum(files, &n);
```

Zložitosť = $O(\log n) + O(\log n) = O(2 \log n) \approx O(\log n)$

3.

```
35 while (i != 0 && files[(i-1)/2] > files[i]) {
```

Zložitosť = $O(\log n)$

4.

```

52 while (n > 1) {
53     // Looking for two files of minimum size:
54     a = minimum(files, &n);
55     b = minimum(files, &n);
56     // Merge the two files into a new file
57 }

```

Zložitosť = $O(n) \times O(\log n) = O(n \log n)$

Celková zložitosť programy = $O(\log n) + O(\log n) + O(n \log n) \approx O(n \log n)$

Two different inputs for your implementation

1 Vstup: {7, 19, 3, 9, 13}

Inicializácia minimálnej haldy: Pôvodné pole sa skonvertuje na minimálnu haldu.

Prvé zlúčenie:

Minimálne prvky: 3 a 7. mergeCost = 3 + 7 = 10. 10 sa vloží späť do haldy: {9, 10, 13, 19}.

Druhé zlúčenie:

Minimálne prvky: 9 a 10. mergeCost = 9 + 10 = 19. 19 sa vloží späť do haldy: {13, 19, 19}.

Tretie zlúčenie:

Minimálne prvky: 13 a 19. mergeCost = 13 + 19 = 32. 32 sa vloží späť do haldy: {19, 32}.

Štvrte zlúčenie:

Minimálne prvky: 19 a 32. mergeCost = 19 + 32 = 51. 51 sa vloží späť do haldy: {51}.

Celkové náklady na zlúčenie: 10 + 19 + 32 + 51 = 112.

2 Vstup: {49, 56, 96, 22, 31, 41}

Inicializácia minimálnej haldy: Pôvodné pole sa skonvertuje na minimálnu haldu.

Prvé zlúčenie:

Minimálne prvky: 22 a 31. mergeCost = 22 + 31 = 53. 53 sa vloží späť do haldy: {41, 49, 96, 53, 56}.

Druhé zlúčenie:

Minimálne prvky: 41 a 49. mergeCost = 41 + 49 = 90. 90 sa vloží späť do haldy: {53, 56, 90, 96}.

Tretie zlúčenie:

Minimálne prvky: 53 a 56. mergeCost = 53 + 56 = 109. 109 sa vloží späť do haldy: {90, 96, 109}.

Štvrte zlúčenie:

Minimálne prvky: 90 a 96. mergeCost = 90 + 96 = 186. 186 sa vloží späť do haldy: {109, 186}.

Päťte zlúčenie:

Minimálne prvky: 109 a 186. mergeCost = 109 + 186 = 295. 186 sa vloží späť do haldy: { 295}.

Celkové náklady na zlúčenie: 53 + 90 + 109 + 186 + 295 = 733.

4.1 Program

Program implementuje Huffmanov algoritmus na efektívne kódovanie znakov na základe ich frekvencie výskytu. Na začiatku sú symboly s ich frekvenciami zoradené podľa rastúcej frekvencie, aby sa vytvoril Huffmanov strom. Po vytvorení stromu sa začína generovanie binárnych kódov pre jednotlivé symboly od koreňa stromu smerom k jeho listom.

Každému symbolu je priradený jedinečný binárny kód, pričom znaky s nižšou frekvenciou dostávajú dlhšie kódy a znaky s vyššou frekvenciou kratšie kódy. Program zobrazuje pre každý symbol jeho frekvenciu, dĺžku kódu a samotný kód.

Na záver program zakóduje reťazec „BE A FAB ACE“ pomocou vygenerovaných kódov pre jednotlivé znaky a výsledok vypíše ako sekvenciu binárnych kódov. Tento proces ukazuje, ako Huffmanov algoritmus efektívne komprimuje dáta.

Môj program má qsort, 3 for, whileforfor a forforfor, pozrime sa na každý z nich, aby sme pochopili zložitosť programu.

1.

```
90 qsort(symbol, n, sizeof(Symbol), compare);  
72 qsort(pocet, m, sizeof(Pocet), compar);
```

Zložitosť = $O(n \log n)$

2.

```
49 while (m > 1) {  
50     sum = pocet[0].frequency + pocet[1].frequency;  
51     //printf("%s ", pocet[0].character);  
52     //printf("%s ", pocet[1].character);  
53     for (i=0; i<n; i++) {  
54         for (j=0; j<20; j++) {  
66         }  
67         strcat(pocet[0].character, pocet[1].character); // napríklad 'AB'  
68         pocet[0].frequency = sum;  
69         m--;  
70         strcpy(pocet[1].character, pocet[m].character);  
71         pocet[1].frequency = pocet[m].frequency;  
72         qsort(pocet, m, sizeof(Pocet), compar);  
32 for (i=29; i>0; i--) {
```

Zložitosť = $O(m) \times (O(n) \times O(20) \times O(30) + O(m \log m)) = O(m) \times (O(n) + O(m \log m)) = O(m) \times O(m \log m) = O(m^2 \log m) \approx O(n^2 \log n)$

Celková zložitosť programy = $O(n \log n) + O(n^2 \log n) \approx O(n^2 \log n)$

4.2 Program

Program implementuje Huffmanov algoritmus na efektívne kódovanie znakov na základe ich frekvencie výskytu. V tomto programe som ako základ zobral 2 pravidlá Huffmanovho algoritmu: Symboly sú pravdepodobnejšie spojené s kratšími kódmi. A tiež znaky s rovnakou frekvenciou sú zoradené podľa abecedy. Môj program skladá kódy, ktoré sú čo najkratšie, no pri kódovaní dáva priestor na oddelenie písmen a slov.

Môj program má qsort, 4 for a forfor, pozrime sa na každý z nich, aby sme pochopili zložitosť programu.

1.

```
54 qsort(symbol, n, sizeof(Symbol), compare);
```

Zložitosť = $O(n \log n)$

2.

```
21 for (i=0; i<m; i++) {
22     code[m-1-i] = (number % 2) + '0';
23     number /= 2;
```

Zložitosť = $O(m) \approx O(n)$

3.

```
31 for (i=0; i<n; i++) {
32     if (i>0 && i == (int)pow(2, length)) {
```

Zložitosť = $O(n)$

4.

```
47 for (i=0; i<n; i++) {
```

Zložitosť = $O(n)$

5.

```
57 for (i=0; i<n; i++) {
```

Zložitosť = $O(n)$

6.

```
64 for (i=0; i<12; i++) {
65     g=0;
66     for (j=0; j<n; j++) {
```

Zložitosť = $O(12) \times O(n) \approx O(12n) \approx O(n)$

Celková zložitosť programy = $O(n \log n) + O(n) + O(n) + O(n) + O(n) + O(n) \approx O(n \log n)$